

Transformers and Large Language Models (LLMs)

Dr. A. V. Brahmane

Department of Computer Engineering

Sanjivani College of Engineering, Kopergaon

Introduction

- Natural Language Processing (NLP) has undergone several revolutions—from rule-based systems to statistical methods, machine learning, deep learning, and finally **Transformers**, which became the foundation of today's **Large Language Models (LLMs)**.
- Before Transformers, models such as **RNNs** and **LSTMs** processed text **one word at a time**. This sequential processing made them slow and unable to effectively capture relationships between words that were far apart in a sentence.
- In 2017, researchers introduced the Transformer architecture through the landmark paper **Attention Is All You Need**.
- The Transformer uses a mechanism called **Self-Attention**, allowing every word in a sentence to interact with every other word simultaneously.
- This innovation dramatically improved language understanding and enabled the creation of powerful Large Language Models such as **ChatGPT**, **Gemini**, **Claude**, and **Llama**.

What is Transformer?

- Transformer is a neural network architecture used for various machine learning tasks, especially in natural language processing and computer vision.
- It focuses on understanding relationships within data to process information more effectively.
- Uses attention mechanisms to capture relationships between inputs.
- Processes entire sequences at once instead of step by step.
- Improves performance on tasks involving context and dependencies.
- Widely used across NLP, vision and other AI applications.

❖ **Word Embedding Techniques in NLP**

Frequency-based Embeddings
Prediction-based Embeddings

❖ **Positional Encoding** is a technique used to add the position (order) of each word in a sentence to its word embedding so that the Transformer knows where each word appears.

❖ **Self-Attention** helps each word understand the importance of other words in the sentence.

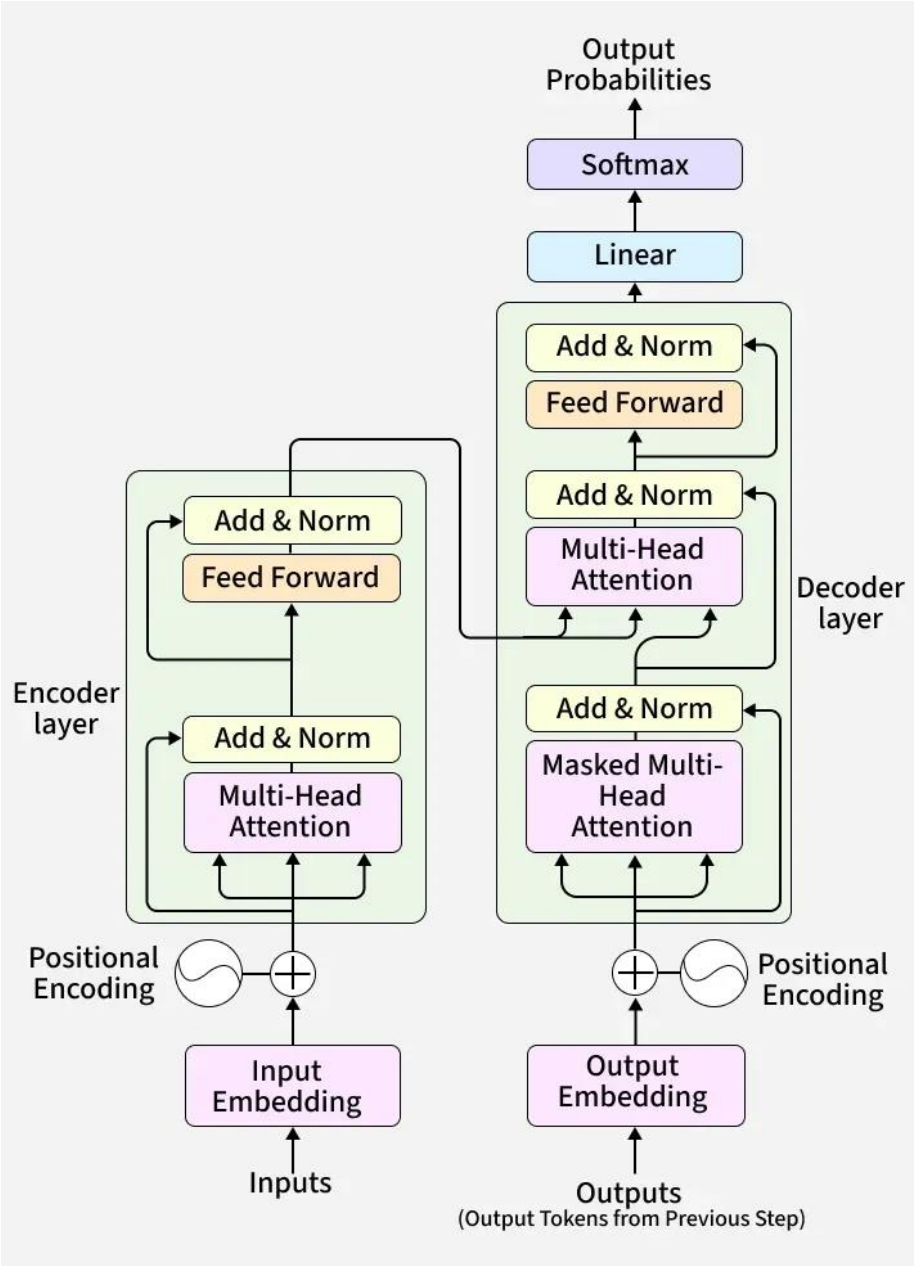
❖ **Add (Residual Connection)**
The original input is added back.
 $\text{Output} = \text{Attention Output} + \text{Original Input}$
Why?
So that important information from the original sentence is not lost.

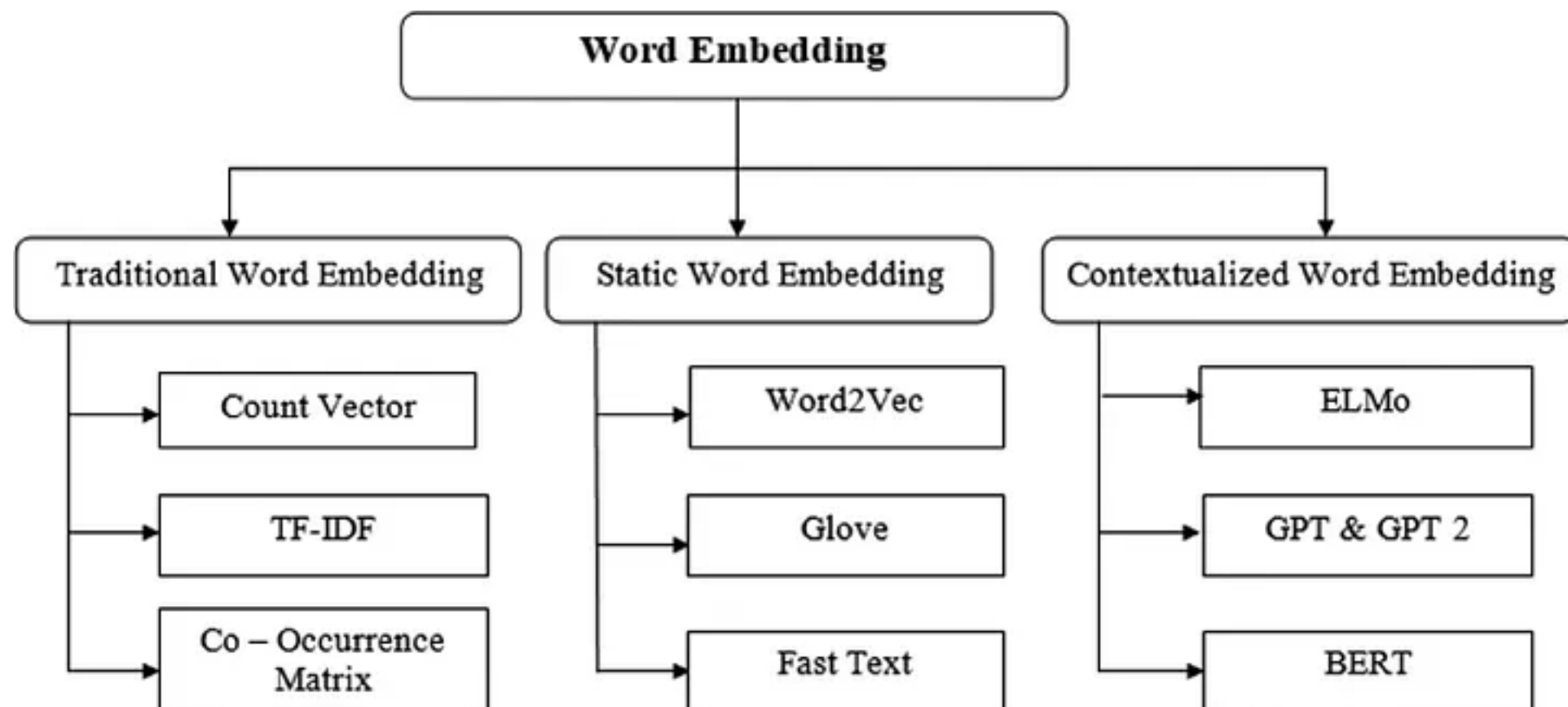
❖ **Layer Normalization**
This keeps the values stable during training.
Think of it as:
Balancing the data so the model learns efficiently.

❖ **Feed Forward Neural Network (FFN)**
Now each word is processed independently by a small neural network.
Its job is to learn **more complex patterns**.

❖ **Add & Layer Normalization (Again)**
Again,
Add the previous information.
Normalize the values.
This improves training stability.

❖ **The Decoder** generates the output sentence one word (or token) at a time by using the previously generated words and the information received from the Encoder.





Frequency-based Word Embedding Technique in NLP

- ***Frequency-based embeddings*** are representations of words in a corpus based on their frequency of occurrence and relationships with other words.
- **TF-IDF and the co-occurrence matrix**
- **TF-IDF (Term Frequency-Inverse Document Frequency)**
- **Term Frequency (TF)**: Measures how often a term occurs in a document. It is calculated as the number of times a term appears in a document divided by the total number of terms in the document.
- **Inverse Document Frequency (IDF)**: Measures how unique a term is across a collection of documents. It is calculated as the logarithm of the total number of documents divided by the number of documents containing the term.
- **TF-IDF Weighting**: The TF-IDF weight of a term in a document is the product of its TF and IDF values. Terms with high TF-IDF weights are considered more important in the context of the document and the corpus.

Co-occurrence Matrix

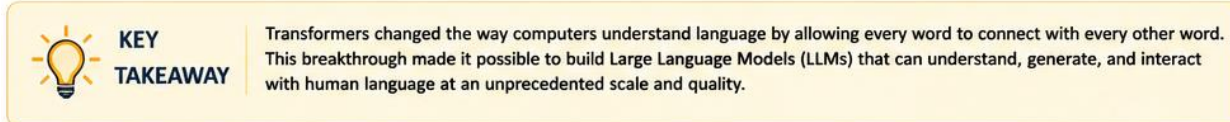
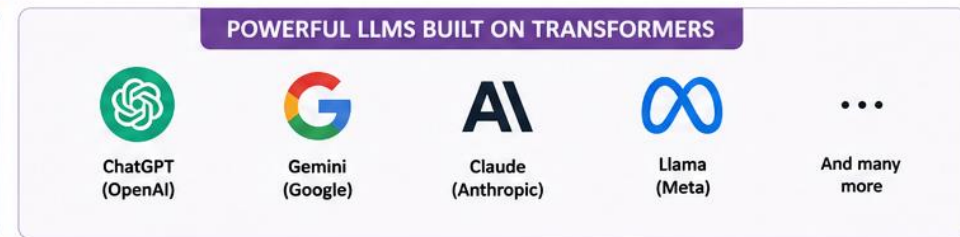
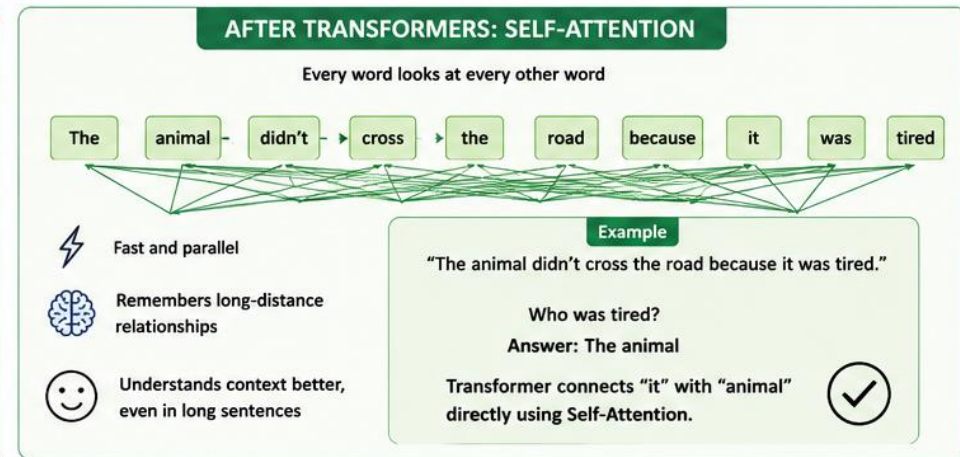
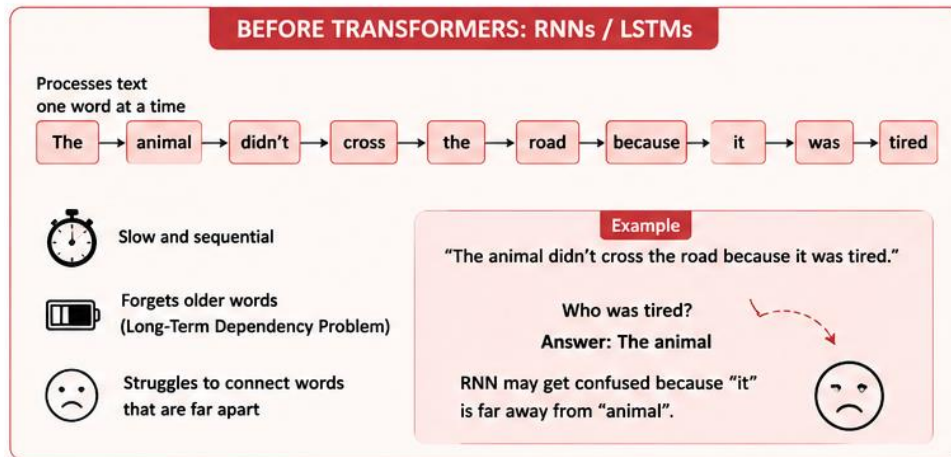
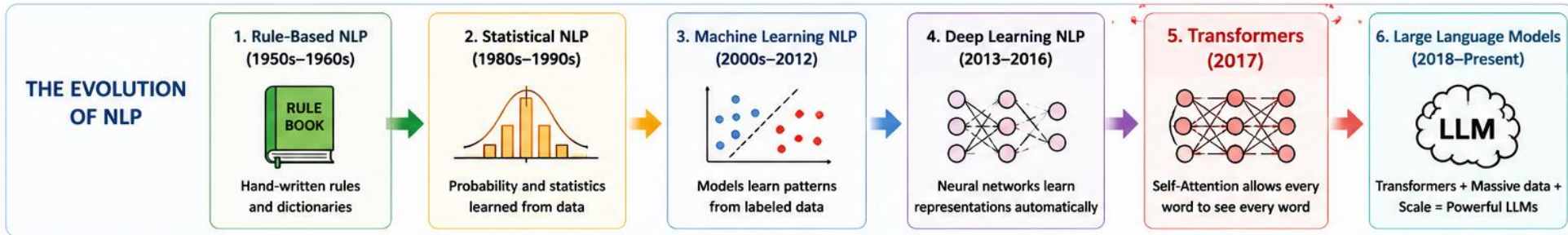
- **Context Window:** In this approach, a context window is defined around each word in a corpus (e.g., a sentence or a paragraph).
- **Co-occurrence Matrix:** A matrix is constructed where rows and columns represent words, and each cell contains the count of how often a pair of words co-occur within the context window.
- **Dimension Reduction:** Techniques like Singular Value Decomposition (SVD) can be applied to reduce the dimensionality of the co-occurrence matrix and capture latent semantic relationships between words.
- **Word Similarity:** The resulting embeddings can be used to measure the similarity between words based on their co-occurrence patterns in the corpus.

Prediction-based Word Embedding Techniques in NLP

- Prediction-based embeddings are generated by training models to predict words in a given context.
- Some popular prediction-based embedding techniques include *Word2Vec (Skip-gram and CBOW)*, *FastText*, and *Global Vectors for Word Representation (GloVe)*.

Transformers and LLMs: The Next Revolution in NLP

From Processing Words One by One to Understanding Everything All at Once



Self-Attention Mechanism

- Self-Attention is a mechanism in the Transformer architecture that enables each word in a sentence to identify and focus on the other words that are most relevant for understanding its meaning.
- *Every word asks, "Which other words in this sentence are important for me?"*
- Instead of reading the sentence one word at a time, every word can "look at" every other word simultaneously.

SELF-ATTENTION MECHANISM IN TRANSFORMERS

Every word looks at every other word and decides what is important!

1. WHY DO WE NEED SELF-ATTENTION?

RNN / LSTM (Old Way)

Reads one word at a time.

The → animal → didn't → ... → it → was → tired

✗ When it reaches "it",
"animal" is far back.



✗ May forget important
words (Long-Term
Dependency Problem).

Transformer with Self-Attention (New Way)

Reads all words at the same time.

The animal didn't cross the road because it was tired



- ✓ "it" can directly look at every other word.
- ✓ Finds "animal" is most relevant.

EXAMPLE SENTENCE

The animal didn't cross the road because it was tired.

Question: Who was tired?



Word	The	animal	didn't	cross	the	road	because	it	was	tired
Attention Score (for "it")	0.01	0.80	0.05	0.03	0.01	0.02	0.04	-	0.02	0.02

Highest attention on "animal" → "it" refers to "animal"

2. HOW SELF-ATTENTION WORKS (STEP BY STEP)

Step 1: Tokenization

Break the sentence into tokens (words).

I love Artificial Intelligence
1 2 3 4

Step 2: Embedding

Convert each token into a vector (numbers).

I → [0.12 0.45 ... 0.33]
love → [0.91 0.32 ... 0.11]
Artificial → [0.54 0.78 ... 0.21]
Intelligence → [0.63 0.44 ... 0.67]
⋮
⋮

Step 3: Create Q, K, V

From each embedding, create three vectors: Query, Key, Value.

Token	Query (Q)	Key (K)	Value (V)
I	[...]	[...]	[...]
love	[...]	[...]	[...]
Artificial	[...]	[...]	[...]
Intelligence	[...]	[...]	[...]
...	...	...	...

Q: What am I looking for?

K: What information do I have?

V: What information do I pass on?

Step 4: Calculate Attention Scores

Each word's Query (Q) is compared with every other word's Key (K).

Example: For the word "love" (row 2)

Compared with (Key)	Similarity (Q · K)	Attention (Softmax)
I	0.20	10%
love	0.80	25%
Artificial	1.10	30%
Intelligence	1.30	35%

Higher similarity → higher attention.
All attention scores add up to 100%.

Step 5: Create New Representation

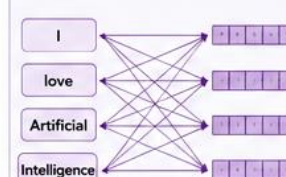
Multiply each Value (V) by its attention score and add them up.

I - 10% → [...]
love - 25% → [...]
Artificial - 30% → [...]
Intelligence - 35% → [...]
New representation for "love"

Now "love" knows information from all other words, with more focus on important ones.

Step 6: Repeat for All Words

The same process is done for every word in the sentence.



This is SELF-ATTENTION

SELF-ATTENTION ON THE EXAMPLE SENTENCE



KEY TAKEAWAY

- Every word looks at every other word.
- It gives more importance (attention) to relevant words and less to others.
- This helps the model understand context, especially in long sentences.
- Self-attention enables parallel processing and better performance than RNNs.

IMPORTANCE



Understands
relationships
better



Faster
(Parallel
Processing)



Handles long
sentences
effectively



Foundation of
Transformers
and LLMs



IN ONE LINE

Self-Attention allows each word to focus on the most important words in the sentence, helping the model understand meaning and context accurately. This simple idea is the foundation of Transformers and Large Language Models like ChatGPT, Gemini, Claude, and Llama.

HOW ATTENTION SCORE IS CALCULATED (Self-Attention) – Simple Example

Example Sentence: “The animal didn’t cross the road because it was tired.”

	1	2	3	4	5	6	7	8	9	10
Tokens	The	animal	didn't	cross	the	road	because	it	was	tired

STEP 1: Create Q, K, V

From each token embedding, we create three vectors: Query (Q), Key (K), Value (V).

Token	Query (Q)	Key (K)	Value (V)
The	[0.1, 0.3]	[0.2, 0.1]	[0.4, 0.2]
animal	[0.3, 0.6]	[0.5, 0.4]	[0.7, 0.1]
didn't	[0.2, 0.1]	[0.1, 0.2]	[0.3, 0.2]
...	...	...	...
it	[0.6, 0.8]	[0.7, 0.6]	[0.6, 0.3]
...	...	...	...

(Numbers are small 2D vectors for easy understanding)

STEP 2: Take the Query of the current word

Let's calculate attention for the word “it” (position 8).

Query (Q) of “it”
[0.6, 0.8]

STEP 3: Compare with Keys of all other words (Similarity Score)

Compute dot product between Q (it) and Key of every word.

Word	Key (K)	Dot Product (Q · K)
The	[0.2, 0.1]	$(0.6 \times 0.2) + (0.8 \times 0.1) = 0.20$
animal	[0.5, 0.4]	$(0.6 \times 0.5) + (0.8 \times 0.4) = 0.62$
didn't	[0.1, 0.2]	$(0.6 \times 0.1) + (0.8 \times 0.2) = 0.22$
cross	[0.3, 0.2]	$(0.6 \times 0.3) + (0.8 \times 0.2) = 0.34$
the	[0.2, 0.1]	$(0.6 \times 0.2) + (0.8 \times 0.1) = 0.20$
road	[0.4, 0.3]	$(0.6 \times 0.4) + (0.8 \times 0.3) = 0.48$
because	[0.2, 0.2]	$(0.6 \times 0.2) + (0.8 \times 0.2) = 0.28$
it	[0.7, 0.6]	$(0.6 \times 0.7) + (0.8 \times 0.6) = 0.90$
was	[0.3, 0.2]	$(0.6 \times 0.2) + (0.8 \times 0.2) = 0.34$
tired	[0.4, 0.1]	$(0.6 \times 0.4) + (0.8 \times 0.1) = 0.32$

STEP 4: Scale (Optional for stability)

Divide by $\sqrt{d_k}$. Here $d_k = 2 \rightarrow \sqrt{2} = 1.414$

Scaled Score = Dot Product / 1.414

Example (animal): $0.62 / 1.414 = 0.44$

Example (it): $0.90 / 1.414 = 0.64$

STEP 5: Apply Softmax to get Attention Scores (Weights)

Softmax converts scaled scores to probabilities (0 to 1) that sum to 1.

Word	Attention Score
The	0.03
animal	0.22
didn't	0.04
cross	0.07
the	0.03
road	0.12
because	0.05
it	0.32
was	0.07
tired	0.05

Total = 1.00

WHAT DOES THIS MEAN?



For the word “it”, the model gives highest attention to “it” itself (0.32) and next highest to “animal” (0.22).



This helps the model understand that “it” refers to “animal”.



Every word in the sentence does the same process (looks at all words and decides what is important).

STEP 6: Get the New Representation (Weighted Sum of Values)

Multiply each Value (V) by its attention score and add them up.

New Representation of “it”
 $= \sum (\text{Attention Score} \times \text{Value})$

High attention words (like “it” and “animal”) contribute more to the final meaning.



SUMMARY:

- 1 Create Q, K, V
- 2 Pick Query of current word
- 3 Compare with all Keys (Dot Product)
- 4 Scale ($\div \sqrt{d_k}$)
- 5 Softmax → Attention Scores (Weights)
- 6 Multiply with Values → New Representation

Why Did We Need Transformers?

- Start with a real-life example.
- Imagine I say:
- *"The boy who won the national robotics competition last year received a scholarship because he worked very hard."*
- Now ask your students:
- **Who worked very hard?**
- Answer: *The boy*
- A computer has to connect "**he**" with "**boy**", even though many words separate them.

Problem with RNN

- RNN reads like this:
- *Word 1 → Word 2 → Word 3 → Word 4 → Word 5 ...*
- It remembers previous words.
- As the sentence becomes longer,
- the memory becomes weaker.
- This is called the
- **Long-Term Dependency Problem**

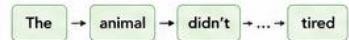
TRANSFORMERS – THE GAME CHANGER IN NLP

Transformers process **all words at the same time** using Self-Attention, understand long-range relationships, and power today's Large Language Models like ChatGPT, Gemini, Claude, Llama, etc.

1. WHY TRANSFORMERS?

RNN / LSTM (Old Way)

Processes words one by one.



- ✗ Slow (sequential)
- ✗ Forgets long-range context
- ✗ Hard to parallelize

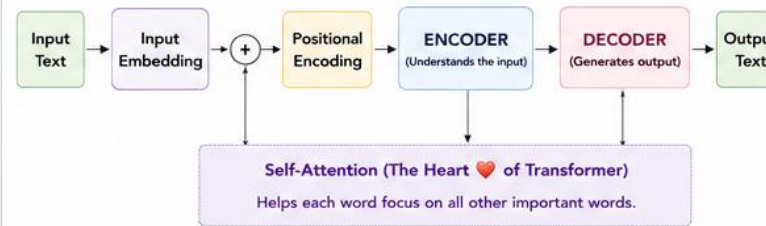
TRANSFORMER (New Way)

Processes all words at the same time.



- ✓ Fast (parallel)
- ✓ Remembers long-range context
- ✓ Handles relationships better

2. THE BIG PICTURE OF TRANSFORMER



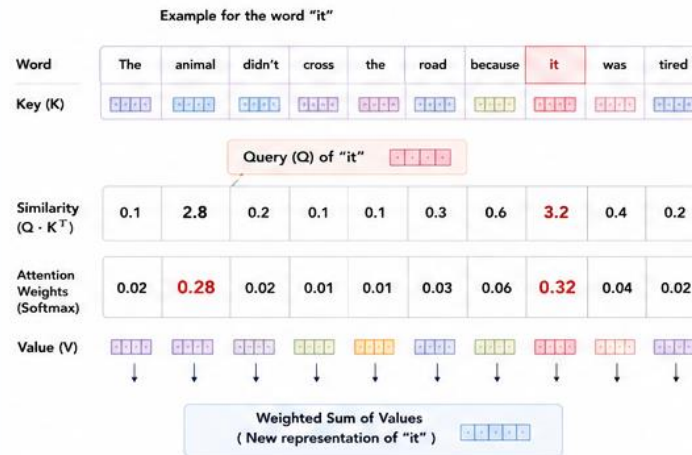
3. WHERE IS TRANSFORMER USED?

- Machine Translation
- Text Summarization
- Question Answering
- Chatbots / Assistants
- Code Generation
- ...and many more!

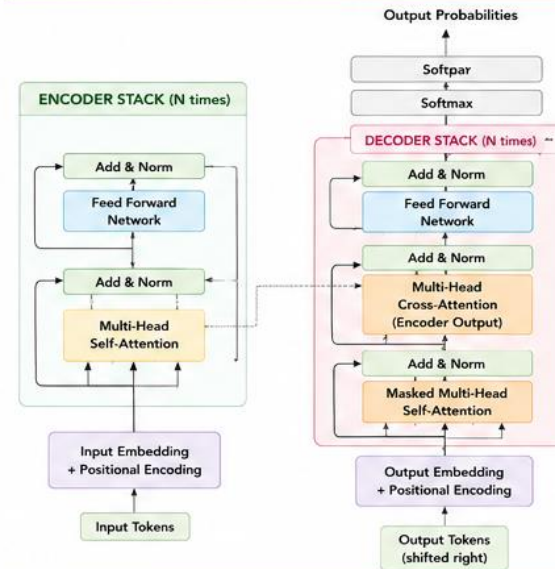
4. HOW SELF-ATTENTION WORKS (Simplified)

Input Sentence: The animal didn't cross the road because it was tired

- Every word is converted into three vectors: Query (Q), Key (K), Value (V) using learned matrices W_Q, W_K, W_V .
- For the word "it", its Query (Q) is compared with the Keys (K) of all words.
- Similarity scores are calculated ($Q \cdot K^T$).
- Scores are scaled and Softmax is applied to get attention weights.
- Each word's Value (V) is multiplied by its attention weight and summed to get the new representation of "it".

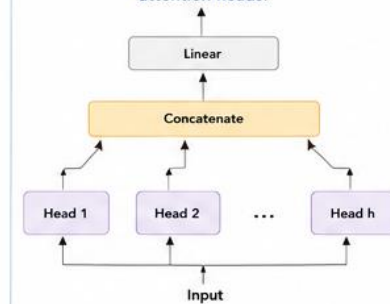


5. TRANSFORMER ARCHITECTURE



6. MULTI-HEAD ATTENTION

Instead of one attention, Transformer uses multiple attention heads.



Multiple heads allow the model to focus on different types of relationships at the same time.

7. POSITIONAL ENCODING – WHY?

Since Transformer sees all words together, it doesn't know their order. Positional Encoding adds position information to embeddings.



8. ADVANTAGES OF TRANSFORMERS

- ✓ Captures long-range dependencies effectively
- ✓ Processes all words in parallel (much faster)
- ✓ Better performance on large datasets
- ✓ Scalable – foundation of LLMs

9. IN ONE LINE

Transformers use Self-Attention to let every word look at all other words, understand the context deeply, and generate powerful, human-like responses.

10. REAL-WORLD IMPACT

ChatGPT, Gemini, Claude, Llama and almost every modern AI tool is built on Transformers.

What are Large Language Models (LLMs)?

A Large Language Model is a Transformer-based neural network trained on enormous amounts of text so that it can understand and generate human language.

Why are They Called Large?

Because they have

- Huge datasets (books, websites, articles, code, research papers)
- Millions or billions of parameters
- Massive computational resources (GPUs/TPUs)
- Long training durations

Why Do LLMs Sound Intelligent?

They learn

- Grammar
- Vocabulary
- Facts (up to their training and available tools)
- Reasoning patterns
- Programming languages
- Writing styles
- Problem-solving strategies

They do **not** think like humans; they **predict the most appropriate next token** based on learned patterns and context.

Popular LLMs

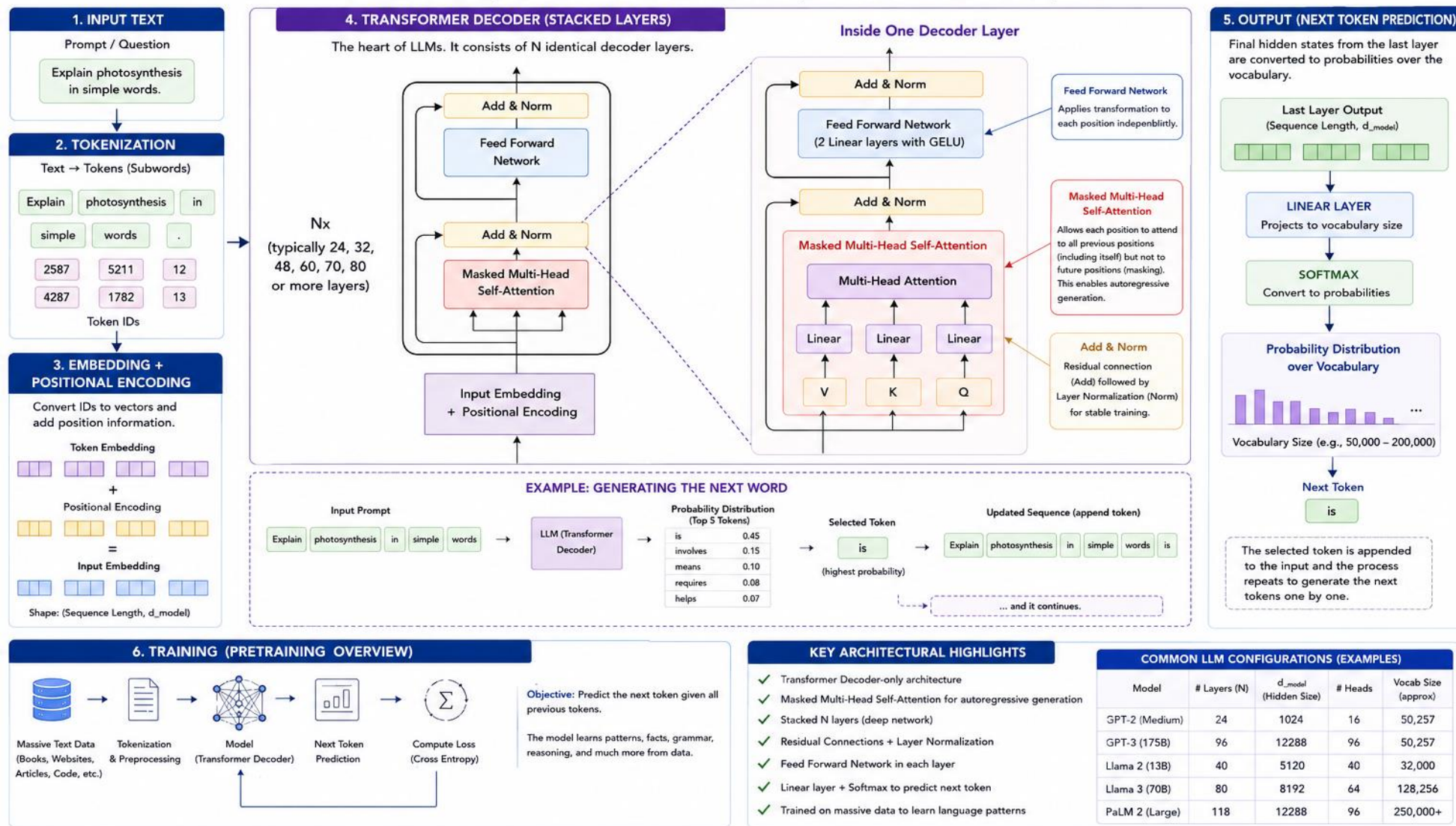
Model	Organization	Major Use
ChatGPT	OpenAI	General-purpose AI assistant
Gemini	Google	Search, multimodal AI
Claude	Anthropic	Long-document reasoning
Llama	Meta	Open-weight AI models

Applications of LLMs

- Intelligent Chatbots
- AI Coding Assistants
- Machine Translation
- Document Summarization
- Content Generation
- Question Answering
- Medical Report Analysis
- Legal Document Analysis
- Education and Tutoring
- Research Assistance

ARCHITECTURE OF A LARGE LANGUAGE MODEL (LLM)

An LLM is basically a Transformer Decoder model that predicts the next token in a sequence.



In One Line

An LLM takes a sequence of tokens, converts them to vectors, processes them through many Transformer decoder layers using masked self-attention, and predicts the most likely next token. This is repeated to generate human-like text.

